

Animación y esqueletos

Eduardo Graells-Garrido

1 Del grafo de escena al cuerpo articulado



Figura 1. Lección de osteología del Dr. Sebastiaen Egbertsz (1619), de Nicolaes Eliaszoon Pickenoy. Que un esqueleto interno gobierne la forma exterior del cuerpo es la idea que organiza la animación de personajes: una jerarquía de huesos conduce la deformación de la piel.

En la unidad de grafos de escena construimos marionetas: objetos jerárquicos donde rotar el hombro arrastra el codo y la mano. Esa estrategia es suficiente para objetos rígidos articulados, como un brazo robótico o las extremidades de un personaje de baja resolución. Sin embargo, en personajes modernos la geometría es una única malla continua que se *deforma* al moverse, de la misma manera que la piel y los músculos de un ser vivo se doblan sin separarse, gobernados por dentro por el esqueleto (Figura 1).

Este es el problema de la **animación de personajes**: dado un esqueleto jerárquico en movimiento, ¿cómo deformamos una malla continua de manera convincente?

En esta unidad acompañamos la teoría con tres ejemplos de la caja de juguetes:

- `esqueleto_lbs` aísla el *skinning* lineal en su forma mínima: un tubo gobernado por una cadena de tres huesos construida a mano, sin cargar ningún archivo. Las teclas rotan dos articulaciones y el tubo se deforma según la fórmula del LBS. El color de la malla mezcla un color por hueso con los mismos pesos del *skinning*, así se ve qué hueso domina cada región.
- `animacion_esqueletica` toma el problema previo al *skinning*: un mecha Valkyrie VF-1S de *Macross* donde cada pieza es rígida y vive como hijo de un hueso del esqueleto. Aquí solo hay jerarquía de transformaciones; no hay pesos por vértice.
- `skinning` toma un zorzal con animación de aleteo y compara **Linear Blend Skinning** (LBS) contra **Dual Quaternion Skinning** (DQS) sobre la misma malla, los mismos huesos y los mismos pesos. La tecla `m` alterna entre ambos métodos en vivo.

Los dos últimos cargan modelos en formato glTF y comparten infraestructura (`grafica.gltf`, `grafica.scenegraph`).

2 El esqueleto: huesos y articulaciones

Un **esqueleto** (*skeleton*) es un grafo de escena especializado. Sus nodos son **articulaciones** (*joints*) y las aristas entre nodos definen **huesos** (*bones*). Cada articulación tiene:

- Una **pose de reposo** (*rest pose* o *bind pose*): la transformación en el espacio local cuando el esqueleto no está animado.
- Una **pose actual**: la transformación aplicada durante la animación.

La jerarquía funciona igual que en un grafo de escena: la transformación de una articulación hija se compone con la de su padre. Si la cadera rota, todo el cuerpo que depende de ella rota con ella.

Espacio de articulación y espacio de objeto

Para animar la malla necesitamos distinguir dos marcos de referencia:

- **Espacio de articulación** (*joint space*): las coordenadas locales de cada hueso, definidas respecto a la articulación padre.
- **Espacio de objeto** (*object/world space*): las coordenadas globales del modelo.

La transformación que lleva un punto desde el espacio de articulación j hasta el espacio de objeto es la **transformación global**:

$$M_j = M_{\text{padre}(j)} \cdot T_j^{\text{local}},$$

calculada desde la raíz del esqueleto hacia las hojas.

3 *Forward Kinematics* (FK)

La **cinemática directa** consiste en especificar la rotación de cada articulación y calcular la posición resultante de los extremos del esqueleto. Es intuitiva para animadores: se rotan hombro, codo y muñeca para posicionar la mano. El proceso sigue el árbol jerárquico de raíz a hoja.

Para cada articulación j con rotación R_j y traslación t_j respecto a su padre:

$$M_j = M_{\text{padre}(j)} \cdot \begin{pmatrix} R_j & t_j \\ 0 & 1 \end{pmatrix}.$$

FK es eficiente y determinista, pero requiere que el animador controle cada grado de libertad del esqueleto de forma explícita.

El ejemplo `animacion_esqueletica` aplica esta idea sin *skinning*: el archivo glTF trae el esqueleto del Valkyrie ya animado por *keyframes* TRS (traslación, rotación, escala) y el código resuelve la jerarquía con el `Scenegraph` que vimos en la unidad de grafos de escena. Cada pieza del robot, parenteada a un hueso, hereda la transformación global de ese hueso. No hay pesos ni deformación: solo composición de matrices (Figura 2).

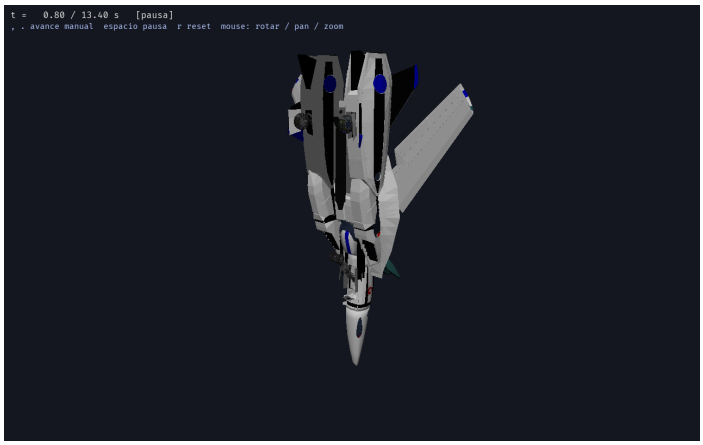


Figura 2. El ejemplo `animacion_esqueletica` del repositorio del curso, capturado a mitad de la transformación de un Valkyrie VF-1S de *Macross*. Cada panel y ala es una pieza rígida parenteada a un hueso del esqueleto; la animación mueve los huesos por *keyframes* TRS y cada pieza hereda la transformación global de su hueso. No hay deformación de malla, solo jerarquía de transformaciones.

4 *Inverse Kinematics* (IK)

La **cinemática inversa** plantea el problema a la inversa: dado un objetivo (por ejemplo, la posición donde debe quedar la mano), ¿qué rotaciones deben tener las articulaciones de la cadena para alcanzarlo? IK es el mecanismo detrás de que un personaje apoye el pie justo sobre un escalón, o que una mano alcance un objeto en cualquier posición.

IK es en general un problema no lineal con múltiples soluciones. Los algoritmos más usados en tiempo real son **CCD** (*Cyclic Coordinate Descent*) y **FABRIK** (*Forward And Backward Reaching Inverse Kinematics*), ambos iterativos y convergentes.

5 *Linear Blend Skinning* (LBS)

El ***skinning* lineal** es el método estándar para deformar una malla mediante un esqueleto. La idea central: cada vértice v de la malla está influenciado por varios huesos, con **pesos** w_j que suman 1. La posición deformada del vértice es una combinación lineal de las posiciones que tendría si solo lo controlase cada hueso:

$$v' = \sum_j w_j \cdot M_j \cdot M_{j,\text{rest}}^{-1} \cdot v.$$

Aquí $M_{j,\text{rest}}^{-1}$ es la **matriz inversa de reposo** (*inverse bind pose matrix*), que lleva el vértice del espacio de objeto al espacio local del hueso j en la pose de reposo. El producto $M_j \cdot M_{j,\text{rest}}^{-1}$ es un viaje de ida y vuelta: la inversa de reposo manda el vértice al espacio local del hueso y la pose actual M_j lo devuelve al mundo, ya animado. En la pose de reposo $M_j = M_{j,\text{rest}}$, así que ese producto vale la identidad y la malla no se deforma. La deformación aparece solo cuando la pose actual se aparta de la de reposo.

Los pesos w_j se almacenan por vértice y se pintan a mano en herramientas como Blender. En la GPU, esta operación se realiza en el *vertex shader*: se pasan las matrices de los huesos como *uniforms* y los pesos como atributos del vértice.

El ejemplo `esqueleto_lbs` muestra esta fórmula sin el ruido de un *loader*. La CPU calcula las matrices $M_j \cdot M_{j,\text{rest}}^{-1}$ por cuadro y el *vertex shader* las mezcla con los pesos. Como el tubo se tinta con la combinación de un color por hueso, las bandas de mezcla se ven doblarse junto al esqueleto, y al doblar mucho una articulación aparece la pérdida de volumen característica del método (Figura 3). La tecla `w` cambia a *wireframe*, donde se ve cómo la triangulación se estira en la cara exterior de la curva y se comprime en la interior (Figura 4).

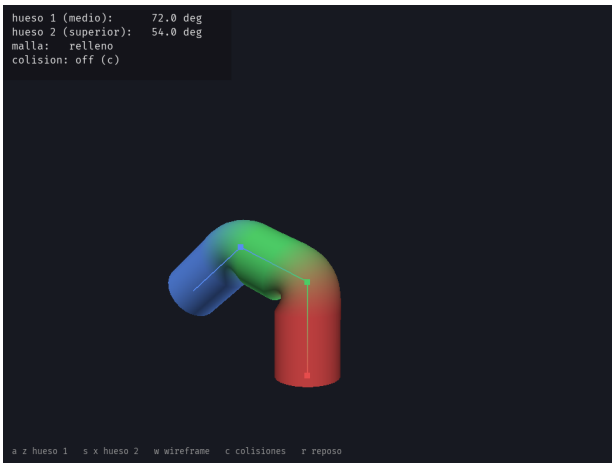


Figura 3. El ejemplo atómico `esqueleto_lbs` del repositorio del curso. Un tubo gobernado por una cadena de tres huesos: la base está fija y dos articulaciones se rotan con el teclado. El esqueleto se dibuja como radiografía sobre la malla, con un color por hueso. El tubo se tinta con la mezcla de esos colores ponderada por los pesos del *skinning*, así las bandas de transición (rojo a verde a azul) se ven doblarse junto al esqueleto.

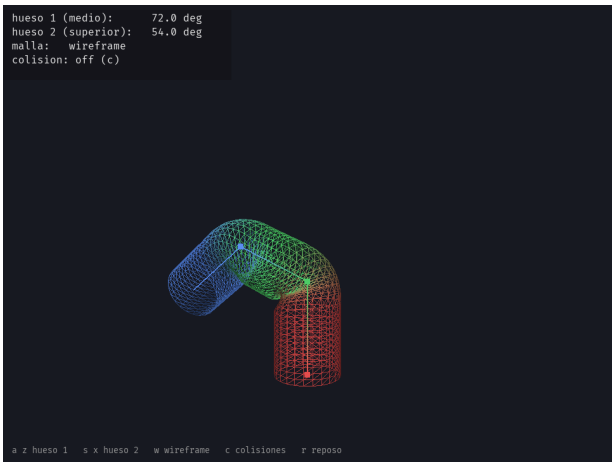


Figura 4. El mismo ejemplo `esqueleto_lbs` en modo *wireframe* (tecla `w`). La triangulación del tubo deja ver la deformación: los triángulos se estiran en la cara exterior de cada curva y se comprimen en la interior.

Las normales también se deforman

La fórmula anterior mueve las posiciones, pero la iluminación necesita las normales y estas no se transforman igual que los puntos. Una normal es una dirección, no una posición: la traslación no la afecta y, cuando la transformación no es una rotación pura, aplicarla tal cual la deja sin ser perpendicular a la superficie. Por eso una normal se transforma con la **matriz de normales**, la inversa traspuesta de la parte lineal 3×3 de la matriz de *skinning*:

$$M_{\text{skin}} = \sum_j w_j \cdot M_j \cdot M_{j,\text{rest}}^{-1}, \qquad n' = \left((M_{\text{skin}})_{3\times 3} \right)^{-T} n.$$

Aquí aparece una sutileza propia del LBS. Aunque cada M_j sea una transformación rígida, la combinación lineal M_{skin} en general no lo es, porque la suma ponderada de rotaciones no es una rotación. Entonces la inversa traspuesta importa: usar

M_{skin} directo sobre la normal la inclina mal justo en las articulaciones, que es donde la mezcla de huesos más se aleja de una rotación. Si no se corrige, el sombreado se ensucia en codos y rodillas aunque la silueta se vea bien. Conviene renormalizar la normal resultante, porque la mezcla cambia su largo.

DQS se ahorra este paso. Como el *dual quaternion* mezclado y renormalizado siempre representa una transformación rígida, la normal se rota con la misma parte real (el cuaternión de rotación) y no hace falta la inversa traspuesta. Es una ventaja secundaria del método, además de conservar mejor el volumen.

Los dos ejemplos de la unidad transforman la normal en el *vertex shader*: `esqueleto_lbs` y la variante LBS de `skinning` construyen la matriz de normales con `transpose(inverse(mat3(skin_matrix)))`, mientras que la variante DQS rota la normal con el cuaternión mezclado.

Limitaciones de LBS

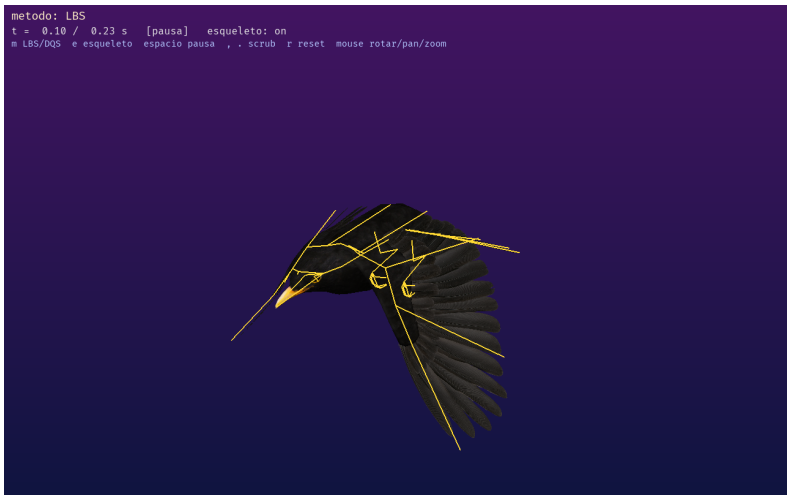
LBS es simple y rápido, pero sufre de artefactos conocidos:

- ***Candy-wrapper artifact***: al torcer una articulación 180°, el volumen colapsa.
- **Pérdida de volumen**: los codos y rodillas pierden volumen al doblarse.

Alternativas como ***Dual Quaternion Skinning* (DQS)** y métodos basados en aprendizaje automático (***Neural Skinning***) reducen estos problemas a costa de mayor complejidad. DQS convierte cada matriz de *skinning* en un par de cuaterniones (parte real y parte dual), los mezcla en la esfera de cuaterniones y renormaliza el resultado: conserva mejor el volumen pero cuesta más por vértice.

El ejemplo `skinning` permite comparar ambos métodos sobre el mismo zorzal. La tecla `e` muestra el esqueleto sobre la malla y la tecla `m` alterna LBS y DQS, lo que ayuda a ver dónde y cuándo aparece el *candy-wrapper* (Figura 5).

Figura 5. El ejemplo `skinning` del repositorio del curso: un zorzal con animación de aleteo, con el esqueleto visible (tecla `e`). La misma malla, los mismos huesos y los mismos pesos se deforman con LBS o con DQS según la tecla `m`. A diferencia del Valkyrie, aquí la malla es continua y cada vértice mezcla la influencia de hasta cuatro huesos.



6 Animación por *keyframes* e interpolación

Una **animación** es una función que asigna a cada instante t una **pose**: el conjunto de transformaciones de todas las articulaciones. En la práctica, los animadores definen poses en instantes discretos llamados ***keyframes*** y el sistema interpola entre ellos.

Interpolación de rotaciones: SLERP

Interpolar rotaciones expresadas como matrices no produce resultados satisfactorios. La solución estándar es usar **cuaterniones de unidad** y la interpolación **SLERP** (*Spherical Linear intERPolation*):

$$\text{slerp}(q_0, q_1, t) = q_0 \cdot \left(q_0^{-1} \cdot q_1\right)^t ,$$

que produce una rotación a velocidad angular constante a lo largo del arco más corto en la esfera unitaria. Para curvas más suaves con múltiples *keyframes* se usa **SQUAD** (*Spherical cubic interpolation*), análogo esférico de las splines cúbicas.

Blend trees

En videojuegos rara vez se reproduce una sola animación a la vez. Los **árboles de mezcla** (*blend trees*) permiten combinar varias animaciones según parámetros continuos: por ejemplo, mezclar “caminar” y “correr” en función de la velocidad, o mezclar “idle” y “apuntar” en la parte superior del cuerpo. Motores como Unity y Unreal incluyen editores visuales de blend trees como parte central de su sistema de animación.

7 Implementación en la GPU

La implementación estándar requiere:

1. **CPU**: calcular las matrices finales de cada hueso ($M_j \cdot M_{j,\text{rest}}^{-1}$) en cada *frame* y cargarlas como un array de matrices *uniform* en el shader.
2. **Vértice en la malla**: almacenar hasta 4 índices de hueso (`joint_ids`) y sus pesos correspondientes (`weights`) como atributos.
3. ***Vertex shader***: mezclar las matrices de los huesos con los pesos y aplicar la matriz resultante a la posición y, vía su matriz de normales, a la normal.

```
// Fragmento de vertex shader con LBS (4 influencias)
mat4 skin_matrix = mat4(0.0);
for (int i = 0; i < 4; i++) {
    skin_matrix += weights[i] * bone_matrices[joint_ids[i]];
}
vec4 skinned_pos = skin_matrix * vec4(position, 1.0);

// la normal usa la matriz de normales (inversa traspuesta de la parte 3×3),
// porque la matriz de skinning mezclada no es una rotacion pura
mat3 normal_matrix = transpose(inverse(mat3(skin_matrix)));
vec3 skinned_normal = normalize(normal_matrix * normal);

gl_Position = projection * view * skinned_pos;
```

El ejemplo `skinning` implementa este patrón en `vertex_program_lbs.glsl` y la variante DQS en `vertex_program_dqs.glsl`. Ambos shaders reciben los mismos atributos por vértice (`joint_ids`, `weights`) y el mismo arreglo de matrices por hueso; la diferencia está solo en cómo combinan esas matrices.

8 Colisiones con mallas deformables

El *skinning* cambia la forma de la malla en cada cuadro, así que toda estructura de colisión calculada una vez sobre la pose de reposo queda inválida apenas se mueve el esqueleto. La AABB del modelo completo deja de contener la malla, una OBB ajustada por PCA a la pose de reposo deja de calzar, y un BVH u Octree sobre los triángulos queda con los límites de sus nodos desactualizados. Es el mismo problema que vimos en la unidad de colisiones (una AABB hay que recalcularla cuando el objeto se transforma), pero ahora distintas partes de la malla se mueven con huesos distintos y no con una sola transformación.

La salida directa, recalcular todo, es cara. Habría que volver a aplicar el *skinning* a todos los vértices en la CPU cada cuadro y reconstruir el volumen o el árbol. Así se paga el costo del *skinning* dos veces, en la GPU para dibujar y en la CPU para colisionar, y reconstruir un árbol por cuadro es costoso. Solo se justifica para un volumen único y grueso.

La solución estándar invierte el enfoque: en vez de encerrar la malla deformada, se pega un volumen rígido a cada hueso. Cada volumen se define en el espacio local de su hueso, por lo que es estático respecto de él, y sigue la transformación global M_j del hueso en cada cuadro. Es el caso de la OBB de la unidad de colisiones (una caja orientada se transforma con el objeto sin recalcularse), pero ahora hay una por hueso y la mueve el esqueleto. El costo es bajo, porque se transforma un puñado de primitivas con las matrices de hueso que el *skinning* ya calcula, no miles de vértices.

La cápsula es la primitiva más usada para esto: un hueso es alargado, una cápsula lo ajusta bien y la prueba cápsula-cápsula es barata. También se usan esferas y OBB. Estos volúmenes por hueso son las *hitboxes* de los videojuegos y la base de los *ragdolls*. Aquí se ve por qué conviene una OBB y no una AABB sobre cada miembro: la OBB rota con el hueso y queda ajustada, mientras que la AABB del mismo miembro hay que recalcularla cada cuadro y se afloja a medida que el miembro se inclina.

El ejemplo `esqueleto_lbs` muestra este contraste con la tecla `c` (Figura 6): dibuja una OBB por hueso, que sigue ajustada al doblar el tubo, junto a la AABB de toda la malla recalculada cada cuadro, que se infla y deja mucho espacio vacío.

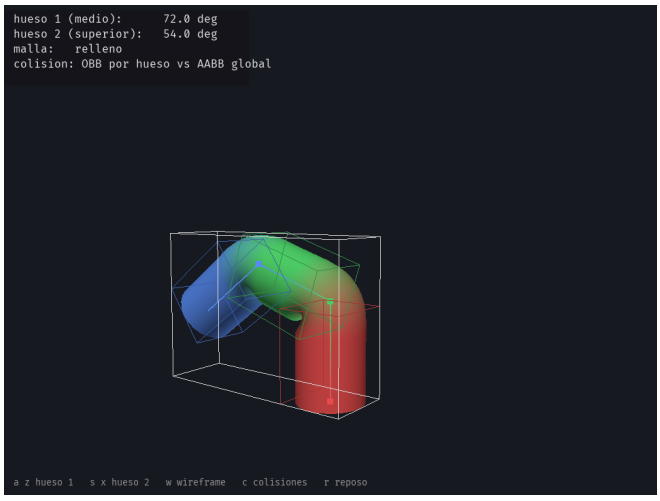


Figura 6. El overlay de colisión del ejemplo `esqueleto_lbs` (tecla `c`). Cada hueso lleva una OBB definida en su espacio local (en los colores del hueso) que se mueve con su pose y queda ajustada al miembro. La caja gris es la AABB de toda la malla, recalculada cada cuadro a partir de los vértices deformados: al doblar el tubo se infla y encierra mucho espacio vacío. Es el *trade-off* AABB contra OBB de la unidad de colisiones, ahora sobre una malla que cambia de forma.

El esquema queda en dos fases, igual que en la unidad de colisiones. Un volumen grueso alrededor de todo el personaje sirve de fase ancha y se arma barato a partir de los extremos de los

volúmenes por hueso; los volúmenes por hueso son la fase angosta.

Si de verdad se necesita colisión a nivel de triángulo contra la superficie deformada, un árbol estático no sirve, pero tampoco hace falta reconstruirlo entero. Un BVH *refittable* conserva la topología del árbol y solo actualiza los límites de cada nodo de abajo hacia arriba en cada cuadro, un *refit* que cuesta $O(n)$ en lugar del costo de reconstruir. Es la técnica habitual para mallas deformables en general, no solo para personajes.

El patrón de fondo es el mismo que en el resto de la unidad: la colisión se apoya en el esqueleto (unos pocos huesos, con sus transformaciones ya calculadas) y no en la malla deformada (miles de vértices). El esqueleto vuelve a ser la estructura barata que gobierna todo.

9 Formatos y herramientas

Los formatos más comunes para exportar mallas con esqueleto son **glTF 2.0** (el estándar web/juegos moderno), **FBX** (ecosistema Autodesk) y **BVH** (*Biovision Hierarchy*, para captura de movimiento). La biblioteca `trimesh` tiene soporte básico para glTF con esqueleto; para animación completa con *skinning* en Python, `pygltflib` da acceso a los datos del esqueleto.

Ambos ejemplos de esta unidad usan glTF 2.0 cargado con la clase `GltfModel` de `grafica.gltf`, que envuelve `pygltflib` y expone los buffers de vértices, las jerarquías de nodos, los canales de animación TRS y, cuando existen, los pesos de *skinning* por vértice junto con las *inverse bind matrices*.

10 Resumen

Concepto	Descripción
Articulación (*joint*)	Nodo del esqueleto con transformación local
Hueso (*bone*)	Arista entre articulaciones padre e hija
FK	Rotaciones → posiciones (de raíz a hoja)
IK	Posición objetivo → rotaciones (iterativo)
LBS	Deformación de malla como combinación lineal ponderada de huesos
Bind pose matrix	Transformación del vértice al espacio del hueso en reposo

Concepto	Descripción
SLERP	Interpolación esférica de cuaterniones
Blend tree	Mezcla continua de múltiples animaciones
Volumen por hueso	Cápsula, OBB o esfera pegada a un hueso; sigue su transformación para colisionar sin re-*skinning*